RA3D Software\ArmController.py

```python
'''This is the armcontroller it controls all elementary moves for the arm. It used the serial controller to send commands
and is used by the print controller to execute printing'''
from asyncio import wait
import numpy as np
import re, time, threading, math, copy
from SerialController import SerialController

class ArmController:
    #region init
    def __init__(self, root, serialController):
        # Save references to the main window and the serial controller
        self.root = root
        self.serialController = serialController
        #Default loop mode Closed
        # Arm calibration variables
        self.armCalibrated = False # Flag to signify if the arm has been calibrated
        self.calibrationOverridden = False # Keeps track of if the calibration was overriden because it can be dangerous
        self.calibrationInProgress = False # Flag to signify if calibration is currently in progress
        # Contains current calibration step
        #  0 - N/A
        #  1 - Send CalStage1
        #  2 - Wait for CalStage1 resposne & process when ready
        #  3 - Send CalStage2
        #  4 - Wait for CalStage2 response & process when ready
        self.calibrationState = 0
        # These variables structure when to calibrate each joint. 1 signifying that joint will be calibrated in that stage
        self.calJStage1 = [1, 1, 1, 0, 0, 0] # J1, J2, & J3 calibration in Stage 1
        self.calJStage2 = [0, 0, 0, 1, 1, 1] # J4, J5, & J6 calibration in Stage 2

        # Speed parameters used for movement commands
        self.defaultMoveParameters = MoveParameters(20,10,10,30,0,'p')
        self.moveParameters = copy.deepcopy(self.defaultMoveParameters)
        self.syncWithPrintParameters = False #Sync basic movements(not all) with the print parameters
        #Movements not affected include move safe move home move origin and jogs

        #J Limits:
        #These are used to determine how red a joint is in the UI
        self.J1Limits = [-170,170]
        self.J2Limits = [-42,90]
        self.J3Limits = [-89,52];
        self.J4Limits = [-165,165]
        self.J5Limits = [-86,105]
        self.J6Limits = [-155,155]

        # Stores current variable information
        # These variables are only used for displaying position not moving
        self.curJ1 = None
        self.curJ2 = None
        self.curJ3 = None
        self.curJ4 = None
        self.curJ5 = None
        self.curJ6 = None
        self.curJ7 = 0

        #Orign variable
        #Note origin is different between arm controller and print controller
        #That is why there is a sync
        self.origin = Origin(None,None,None)

        #Current position of the arm that is received from the Teensy
        self.curPos = Position(None,None,None,None,None,None,self.origin)

        # Stores calibration offset values
        self.J1CalOffset = 0
        self.J2CalOffset = 0
        self.J3CalOffset = 0
        self.J4CalOffset = 0
        self.J5CalOffset = 0
        self.J6CalOffset = 0

        # Flag varables used indicate busyness
        self.awaitingMoveResponse = False # Flag for if the ArmController is awaiting a serial response after sending a move command
        self.testingLimitSwitches = False # Flag for if the limit switch test is being performed
```

```
        self.testingEncoders = False      # Flag for if the encoder test is being performed
        #self.awaitingTestResponse = False # Flag for if we are awaiting a response after sending a test command such as for the limit switches c
        self.finishTest = False           # Flag for signifying to the program that the user wants to stop a test
        self.awaitingPosResponse = False  # Flag for requesting current position

        #Tool jog variables
        self.V1 = 1 #axis 1
        self.V2 = 1 #positive

        #Extruder variables
        self.extruder_deg_per_mm_cool = 10.90909
        self.heatedFilamentMultiplier = 0.7 #multiplier for extrusion when filament is heated, determined experimentally
        self.extruder_deg_per_mm = self.extruder_deg_per_mm_cool * self.heatedFilamentMultiplier
        self.loadLength = 450 #length of filament to load fst, determined experimentally
        self.defaultExtrudeParameters = MoveParameters(30,10,10,30,0,'m')
        self.unloadParameters = MoveParameters(80,10,10,30,0,'p')

    #endregion init

    #region ========|Calibration|==========

    #Initialized arm calibration
    #check if calibration is possible then start calibration thread
    def startArmCalibration(self):
        # Check if calibration is already in progress and exit if so
        #TODO a warning print is printed for all buttons when board is not connected
        if not self.serialController.boardConnected:
            self.root.warningPrint("Board not connected")
            return
        if self.calibrationInProgress is True:
            self.root.statusPrint("Calibration already in progress")
            return
        # Check if arm is busy with something else
        if self.checkIfBusy(message="calibrate arm"):
            return
        #Cannot calibrate while print is paused
        if self.root.printController.checkIfPrinterBusy("calibrate arm"):
            return
        #Note here we did not do a nominal check because that checks if the arm is calibrated
        self.root.statusPrint("Beginning arm calibration")
        # Set flag for calibration in progress
        self.calibrationInProgress = True
        # Move to next state of calibration
        self.calibrationState = 1
        #clear queue to get rid of any error commands
        self.root.serialController.clearQueue()
        #On every calibration reset to recomended
        self.setOrigin(origin=self.root.printController.recommendedOrigin)

        # Call the calibration update function
        calibrationThread = threading.Thread(target=self.calibrateArm, name="Arm Calibration Thread")
        calibrationThread.start()


    #Calibrate arm which is run as a thread
    #Calibration will wait for response to continue
    def calibrateArm(self):
        response = None
        #Exit the function if calibration is not in progress and exit if so
        #This loop means any function can cancel the calibration simply by setting calibrationInProgress to false
        while self.calibrationInProgress:
            # Check current state and perform associated tasks
            if self.calibrationState == 1: # Send CalStage1
                self.calibrateJoints(calJ1=self.calJStage1[0],
                                     calJ2=self.calJStage1[1],
                                     calJ3=self.calJStage1[2],
                                     calJ4=self.calJStage1[3],
                                     calJ5=self.calJStage1[4],
                                     calJ6=self.calJStage1[5],single=False)
                self.calibrationState = 2
                response = self.serialController.waitForResponse("POSLL",35)
                #reevalute if calibration is in progress
            elif self.calibrationState == 2: # Await CalStage1 Response & process when ready
                # Check if the serial controller has a response ready
                if response is not None:
                    # Save the response
```

```
                    # Check if the calibration was successful
                    # Inform user of Stage 1 success
                    self.root.statusPrint("Stage 1 Calibration Successful")
                    # Process position response and dispaly
                    self.processPosition(response)
                    # Move to next state
                    self.calibrationState = 3
                    # Print out the response received
                    if self.root.DebugMode:
                        self.root.terminalPrint(response)
                    response = None #reset response for next response
                else:
                    # If not, inform user of Stage 1 Failure
                    self.root.statusPrint(f"Stage 1 Calibration FAILED")
                    self.root.terminalPrint("Calibration timed out")
                    # Exit calibration
                    self.calibrationState = 0
                    self.calibrationInProgress = False
                    # Force arm calibration flag to False
                    self.armCalibrated = False
            elif self.calibrationState == 3: # Send CalStage2
                self.calibrateJoints(calJ1=self.calJStage2[0],
                                     calJ2=self.calJStage2[1],
                                     calJ3=self.calJStage2[2],
                                     calJ4=self.calJStage2[3],
                                     calJ5=self.calJStage2[4],
                                     calJ6=self.calJStage2[5],single=False)
                self.calibrationState = 4
                #Calibration will not continue until a response is received or timed out
                response = self.serialController.waitForResponse("POSLL",35)
                #reevaluate calibration in progress
            elif self.calibrationState == 4: # Await CalStage2 Response & process when ready
                # Check if the serial controller has a response ready
                if response is not None:
                    # Check if the calibration was successful
                    # Inform user of Stage 1 success
                    self.root.statusPrint("Stage 2 Calibration Successful")
                    # Process position response and dispaly
                    self.processPosition(response)


                    # Set arm calibration flag to True
                    self.armCalibrated = True
                    # Print out the response received
                    if self.root.DebugMode:
                        self.root.terminalPrint(response)
                else:
                    # If not, inform user of Stage 2 Failure
                    self.root.statusPrint("Stage 2 Calibration FAILED")
                    self.root.terminalPrint("Calibration timed out")
                    # Force arm calibration flag to False
                    self.armCalibrated = False
                #Exit calibration
                # Calibration complete
                self.calibrationState = 0
                # Calibration no longer in progress
                self.calibrationInProgress = False

        #If exited early still set state back to 0
        self.calibrationState = 0

    #Override Calibration if the arm is already calibrate but program restarted
    def overrideCalibration(self):
        #Set the origin
        self.setOrigin(origin=self.root.printController.recommendedOrigin)
        #warn the user
        self.root.warningPrint("Calibration overridden. Only use if you know that arm is calibrated")
        #arm is now considered calibrated
        self.armCalibrated = True
        #Keep track if calibration was overriden
        self.calibrationOverridden = True

    #calibrate indvidual joints
    def calibrateJoints(self, calJ1=False, calJ2=False, calJ3=False, calJ4=False, calJ5=False, calJ6=False, single=True):
        self.getCalOffsets() # Update calibration offsets from entry fields
        #build teensy command
```

```
        command = f"LLA{calJ1}B{calJ2}C{calJ3}D{calJ4}E{calJ5}F{calJ6}G0H0I0J{self.J1CalOffset}K{self.J2CalOffset}L{self.J3CalOffset}M{self.J4Ca

        #Debug log
        if self.root.DebugMode:
            self.root.terminalPrint("Command to send: ")
            self.root.terminalPrint(command[0:-2])#don't include new line character

        #Check if board is connected
        if self.serialController.boardConnected is False:
            self.root.statusPrint("Command not sent due to no board connected")

        # Tell the serial controller to send the serial
        self.serialController.sendSerial(command)

        if single:
            self.waitForResponseAndProcess("POSLL",timeout=30)

    #Post calibrate individual joints
    def postCalibrateJoints(self, calJ1=False, calJ2=False, calJ3=False, calJ4=False, calJ5=False, calJ6=False):
        if self.checkIfAllBusy(message="post calibrate"):
            return
        self.getPostCalOffsets() # Update calibration offsets from entry fields
        #Move home to avoid hitting limit switches and to properly measure calibration with level
        self.moveHome()
        #Prep command
        command = f"LEA{calJ1}B{calJ2}C{calJ3}D{calJ4}E{calJ5}F{calJ6}G0H0I0J{self.J1CalOffset}K{self.J2CalOffset}L{self.J3CalOffset}M{self.J4Ca
        if self.root.DebugMode:
            self.root.terminalPrint("Command to send: ")
            self.root.terminalPrint(command[0:-2]) #don't include new line character

        # Tell the serial controller to send the serial
        self.serialController.sendSerial(command)

        # Set values back to zero to avoid unintentionally stacking offsets
        self.root.J1OffsetEntryP.delete(0, 'end')
        self.root.J2OffsetEntryP.delete(0, 'end')
        self.root.J3OffsetEntryP.delete(0, 'end')
        self.root.J4OffsetEntryP.delete(0, 'end')
        self.root.J5OffsetEntryP.delete(0, 'end')
        self.root.J6OffsetEntryP.delete(0, 'end')
        self.root.J1OffsetEntryP.insert(0,"0")
        self.root.J2OffsetEntryP.insert(0,"0")
        self.root.J3OffsetEntryP.insert(0,"0")
        self.root.J4OffsetEntryP.insert(0,"0")
        self.root.J5OffsetEntryP.insert(0,"0")
        self.root.J6OffsetEntryP.insert(0,"0")
        self.waitForResponseAndProcess("POSLE",timeout=30)

    #Get post cal offsets from entry fields
    def getPostCalOffsets(self):
        self.J1CalOffset = float(self.root.J1OffsetEntryP.get())
        self.J2CalOffset = float(self.root.J2OffsetEntryP.get())
        self.J3CalOffset = float(self.root.J3OffsetEntryP.get())
        self.J4CalOffset = float(self.root.J4OffsetEntryP.get())
        self.J5CalOffset = float(self.root.J5OffsetEntryP.get())
        self.J6CalOffset = float(self.root.J6OffsetEntryP.get())
        if self.root.DebugMode:
            self.root.terminalPrint(f"Post calibration offsets J1: {self.J1CalOffset}, J2: {self.J2CalOffset}, J3: {self.J3CalOffset}, J4: {self.

    # get cal offesets from entry fields
    def getCalOffsets(self):
        # Grab values from the entry fields, convert to integers, and save
        self.J1CalOffset = float(self.root.J1OffsetEntry.get())
        self.J2CalOffset = float(self.root.J2OffsetEntry.get())
        self.J3CalOffset = float(self.root.J3OffsetEntry.get())
        self.J4CalOffset = float(self.root.J4OffsetEntry.get())
        self.J5CalOffset = float(self.root.J5OffsetEntry.get())
        self.J6CalOffset = float(self.root.J6OffsetEntry.get())
        if self.root.DebugMode:
            self.root.terminalPrint(f"Calibration offsets J1: {self.J1CalOffset}, J2: {self.J2CalOffset}, J3: {self.J3CalOffset}, J4: {self.J4Ca
    #endregion Calibration

    #region ========|Position|=============

    #Process position response from teensy to update UI
    def processPosition(self, response):
```

```
        if self.root.DebugMode:
            self.root.terminalPrint(response)
        AIdx = response.find('A') # A value is angle of J1
        response = response[AIdx:] # Remove POS from string
        # Collect all the indexes for finding values
        # General formatting of a response: A[val]B[val]C[val]D[val]E[val]F[val]G[val]H[val]I[val]J[val]K[val]L[val]M[val]N[val]O[val]P[val]Q[va
        J1Idx = response.find('A') # A value is angle of J1
        J2Idx = response.find('B') # B value is angle of J2
        J3Idx = response.find('C') # C value is angle of J3
        J4Idx = response.find('D') # D value is angle of J4
        J5Idx = response.find('E') # E value is angle of J5
        J6Idx = response.find('F') # F value is angle of J6
        XPosIdx = response.find('G') # G value is X position
        YPosIdx = response.find('H') # H value is Y position
        ZPosIdx = response.find('I') # I value is Z position
        RzIdx = response.find('J') # J value is Rz rotation
        RyIdx = response.find('K') # K value is Ry rotation
        RxIdx = response.find('L') # L value is Rx rotation
        SpeedViolationIdx = response.find('M') # M value is if there is a speed violation
        DebugIdx = response.find('N') # N value is ???
        FlagIdx = response.find('O') # O value is ???
        J7Idx = response.find('P') # P value is angle of J7
        J8Idx = response.find('Q') # Q value is angle of J8
        J9Idx = response.find('R') # R value is angle of J9

        # Extract the actual values from the response
        # Joint angles
        self.curJ1 = float(response[J1Idx+1:J2Idx].strip())
        self.curJ2 = float(response[J2Idx+1:J3Idx].strip())
        self.curJ3 = float(response[J3Idx+1:J4Idx].strip())
        self.curJ4 = float(response[J4Idx+1:J5Idx].strip())
        self.curJ5 = float(response[J5Idx+1:J6Idx].strip())
        self.curJ6 = float(response[J6Idx+1:XPosIdx].strip())

        # XYZ Positions
        self.curPos.x = float(response[XPosIdx+1:YPosIdx].strip())
        self.curPos.y = float(response[YPosIdx+1:ZPosIdx].strip())
        self.curPos.z = float(response[ZPosIdx+1:RzIdx].strip())

        # RXYZ Angles
        self.curPos.Rx = float(response[RxIdx+1:SpeedViolationIdx].strip())
        self.curPos.Ry = float(response[RyIdx+1:RxIdx].strip())
        self.curPos.Rz = float(response[RzIdx+1:RyIdx].strip())
        #print("J7string",response[J7Idx+1:J8Idx].strip())
        self.curJ7 = round(float(response[J7Idx+1:J8Idx].strip())/self.extruder_deg_per_mm,2)

        #There are 2 displays in the program
        self.root.currentJ7.config(text=f"Extruded: {self.curJ7} mm")
        self.root.currentJ72.config(text=f"Extruded: {self.curJ7} mm")

        # Display values on UI
        # XYZ
        self.root.xCurCoord.config(text=self.curPos.x)
        self.root.yCurCoord.config(text=self.curPos.y)
        self.root.zCurCoord.config(text=self.curPos.z)
        self.root.RxCurCoord.config(text=self.curPos.Rx)
        self.root.RyCurCoord.config(text=self.curPos.Ry)
        self.root.RzCurCoord.config(text=self.curPos.Rz)
        jointColors=self.getJointColors(self.curJ1,self.curJ2,self.curJ3,self.curJ4,self.curJ5,self.curJ6)
        # Joint
        self.root.J1CurCoord.config(text=self.curJ1, fg=jointColors[0])
        self.root.J2CurCoord.config(text=self.curJ2, fg=jointColors[1])
        self.root.J3CurCoord.config(text=self.curJ3, fg=jointColors[2])
        self.root.J4CurCoord.config(text=self.curJ4, fg=jointColors[3])
        self.root.J5CurCoord.config(text=self.curJ5, fg=jointColors[4])
        self.root.J6CurCoord.config(text=self.curJ6, fg=jointColors[5])
        self.updateDeltaFromOrigin() #update delta from origin display

    #Position is updated without any arm movement
    def requestPositionManual(self):
        #Check if busy
        if self.checkIfBusy(message="request position"):
            return
        #start position thread
        positionThread = threading.Thread(target=self.requestPositionAndWait, name="Request Position Thread")
        positionThread.start()
```

```
378
379     #position request is sent and recieved
380     #Needs to be run as a thread if not already on a thread
381     def requestPositionAndWait(self):
382         self.root.statusPrint("Requesting position update...")
383         self.serialController.sendSerial("RP\n") # Send instruction
384         self.awaitingPosResponse = True # Set the flag
385         response = self.serialController.waitForResponse("POSRP",7)
386 
387         # Inform user and process the position response
388         if response is not None:
389             self.root.statusPrint("Position request fulfilled")
390             self.processPosition(response)
391         # Reset the awaiting position respone flag
392         self.awaitingPosResponse = False
393 
394     #endregion process position
395 
396     #region GUI Functions
397 
398     #Verify whether post calibration is possible then start thread
399     def startPostCalibration(self, calJ1=False, calJ2=False, calJ3=False, calJ4=False, calJ5=False, calJ6=False):
400         # Check if calibration is already in progress and exit if so
401         if self.calibrationInProgress:
402             self.root.statusPrint("Calibration already in progress")
403             return
404         # Check if arm is busy with something else
405         if self.checkIfBusy():
406             self.root.statusPrint("Arm is busy with something else at the moment")
407             return
408         self.root.statusPrint("Beginning post calibration")
409         postCalibrateThread = threading.Thread(target=self.postCalibrateJoints, kwargs={"calJ1":calJ1,"calJ2":calJ2, "calJ3":calJ3, "calJ4":calJ4
410         postCalibrateThread.start()
411         #self.postCalibrateJoints(calJ1=calJ1, calJ2=calJ2, calJ3=calJ3, calJ4=calJ4, calJ5=calJ5, calJ6=calJ6)
412 
413     #calibration for one joint at a time
414     def startSpecificCalibration(self, calJ1=False, calJ2=False, calJ3=False, calJ4=False, calJ5=False, calJ6=False):
415         # Check if calibration is already in progress and exit if so
416         if self.calibrationInProgress:
417             self.root.statusPrint("Calibration already in progress")
418             return
419         # Check if arm is busy with something else
420         if self.checkIfBusy():
421             self.root.statusPrint("Arm is busy with something else at the moment")
422             return
423         self.root.statusPrint("Beginning arm calibration")
424         specificCalibrationThread = threading.Thread(target=self.calibrateJoints, kwargs={"calJ1":calJ1,"calJ2":calJ2, "calJ3":calJ3, "calJ4":cal
425         specificCalibrationThread.start()
426 
427 
428     #getxyz from teensy and populate entry fields
429     def populateMJ(self):
430         if self.checkIfBusy():
431             self.root.statusPrint("Failed to request position update. Arm is busy.")
432             return
433         #Get latest position
434         self.requestPositionAndWait()
435         #Delete current entries
436         self.root.xCoordEntry.delete(0, 'end')
437         self.root.yCoordEntry.delete(0, 'end')
438         self.root.zCoordEntry.delete(0, 'end')
439         self.root.RxCoordEntry.delete(0, 'end')
440         self.root.RyCoordEntry.delete(0, 'end')
441         self.root.RzCoordEntry.delete(0, 'end')
442         #Insert current position into entry
443         self.root.xCoordEntry.insert(0,str(self.curPos.x))
444         self.root.yCoordEntry.insert(0,str(self.curPos.y))
445         self.root.zCoordEntry.insert(0,str(self.curPos.z))
446         self.root.RxCoordEntry.insert(0,str(self.curPos.Rx))
447         self.root.RyCoordEntry.insert(0,str(self.curPos.Ry))
448         self.root.RzCoordEntry.insert(0,str(self.curPos.Rz))
449 
450     #get joints from teensy and populate entry fields
451     def populateJoints(self):
452         if self.checkIfBusy():
453             self.root.statusPrint("Failed to request position update. Arm is busy.")
```

```
            return
        self.requestPositionAndWait()
        #delete current entries
        self.root.J1CoordEntry.delete(0, 'end')
        self.root.J2CoordEntry.delete(0, 'end')
        self.root.J3CoordEntry.delete(0, 'end')
        self.root.J4CoordEntry.delete(0, 'end')
        self.root.J5CoordEntry.delete(0, 'end')
        self.root.J6CoordEntry.delete(0, 'end')
        #Insert entries
        self.root.J1CoordEntry.insert(0,str(self.curJ1))
        self.root.J2CoordEntry.insert(0,str(self.curJ2))
        self.root.J3CoordEntry.insert(0,str(self.curJ3))
        self.root.J4CoordEntry.insert(0,str(self.curJ4))
        self.root.J5CoordEntry.insert(0,str(self.curJ5))
        self.root.J6CoordEntry.insert(0,str(self.curJ6))

    #Get RJ entries and start a moveRJ (Rotate Joints) command
    def prepRJCommand(self):
        if self.checkIfAllBusy():
            self.root.statusPrint("Cannot send RJ command. Arm is busy.")
            return
        self.sync() #check if to sync with printer
        # Read the values from each entry box
        J1 = self.root.J1CoordEntry.get()
        J2 = self.root.J2CoordEntry.get()
        J3 = self.root.J3CoordEntry.get()
        J4 = self.root.J4CoordEntry.get()
        J5 = self.root.J5CoordEntry.get()
        J6 = self.root.J6CoordEntry.get()
        # Check if any values are blank
        allValuesNumeric = True
        pattern = r"^-?(\d+(?:\.\d+)?)"  # Regular expression for a valid float

        #If pattern match failed for any joint
        if not re.match(pattern, J1):
            self.root.terminalPrint("J1 is not a number")
            allValuesNumeric = False
        if not re.match(pattern, J2):
            self.root.terminalPrint("J2 is not a number")
            allValuesNumeric = False
        if not re.match(pattern, J3):
            self.root.terminalPrint("J3 is not a number")
            allValuesNumeric = False
        if not re.match(pattern, J4):
            self.root.terminalPrint("J4 is not a number")
            allValuesNumeric = False
        if not re.match(pattern, J5):
            self.root.terminalPrint("J5 is not a number")
            allValuesNumeric = False
        if not re.match(pattern, J6):
            self.root.terminalPrint("J6 is not a number")
            allValuesNumeric = False

        #If match succesful for all
        if allValuesNumeric:
            self.root.terminalPrint("All values numeric, sending RJ command")
            #Start RJ thread
            RJThread = threading.Thread(target=self.sendRJ, args=[[J1, J2, J3, J4, J5, J6], self.moveParameters], name="RJ Command Thread")
            RJThread.start()
        else:
            self.root.terminalPrint("RJ command not sent due to a value not being a number")

    #user action to send ML(Move Linear) command
    #starts a sendML thread
    def prepMLCommand(self):

        #All busy is used to consider printing because this is a user action
        #In general internal commands should not be blocked by printing states but user commands should
        if self.checkIfAllBusy(message="Send ML"):
            return
        self.sync() #check if to sync with printer
        # Read the values from each entry box
        x  = self.root.xCoordEntry.get()
        y  = self.root.yCoordEntry.get()
        z  = self.root.zCoordEntry.get()
```

```
        Rx = self.root.RxCoordEntry.get()
        Ry = self.root.RyCoordEntry.get()
        Rz = self.root.RzCoordEntry.get()
        J7 = self.root.J7CoordEntry.get()
        # Check if any values are blank
        pattern = r"^-?(\d+(?:\.\d+)?)"  # Regular expression for a valid float
        allValuesNumeric = True
        #Check if pattern matches
        if not re.match(pattern, x):
            self.root.terminalPrint("X is not a number")
            allValuesNumeric = False
        if not re.match(pattern, y):
            self.root.terminalPrint("Y is not a number")
            allValuesNumeric = False
        if not re.match(pattern, z):
            self.root.terminalPrint("Z is not a number")
            allValuesNumeric = False
        if not re.match(pattern, Rx):
            self.root.terminalPrint("Rx is not a number")
            allValuesNumeric = False
        if not re.match(pattern, Ry):
            self.root.terminalPrint("Ry is not a number")
            allValuesNumeric = False
        if not re.match(pattern, Rz):
            self.root.terminalPrint("Rz is not a number")
            allValuesNumeric = False

        #Command will still send if J7 is empty
        if not re.match(pattern, J7):
            J7 = 0
        else:
            #Conver to float
            J7 = float(J7)

        #If match successful for all
        if allValuesNumeric:
            #self.root.terminalPrint("All values numeric, sending ML command")
            commandPos = Position(x,y,z,Rx,Ry,Rz,None)
            self.root.statusPrint("Sending ML command")
            #Thread so that command doesn't interupt UI
            MLThread = threading.Thread(target=self.sendML, args=[commandPos, self.moveParameters], kwargs={"extrudeRate":J7,"RelativeExtrude":Tr
            MLThread.start()
        else:
            self.root.statusPrint("ML command not sent due to a value not being a number")

    #Prep moveMJ (Move joints)
    #MJ moves to a xyz coordinate with any path (non-linear)
    def prepMJCommand(self):
        if self.checkIfAllBusy():
            self.root.statusPrint("Cannot send RJ command. Arm is busy.")
            return
        self.sync() #check if to sync with printer
        # Read the values from each entry box
        x  = self.root.xCoordEntry.get()
        y  = self.root.yCoordEntry.get()
        z  = self.root.zCoordEntry.get()
        Rx = self.root.RxCoordEntry.get()
        Ry = self.root.RyCoordEntry.get()
        Rz = self.root.RzCoordEntry.get()
        # Check if any values are blank
        pattern = r"^-?(\d+(?:\.\d+)?)"  # Regular expression for a valid float
        allValuesNumeric = True
        if not re.match(pattern, x):
            self.root.terminalPrint("X is not a number")
            allValuesNumeric = False
        if not re.match(pattern, y):
            self.root.terminalPrint("Y is not a number")
            allValuesNumeric = False
        if not re.match(pattern, z):
            self.root.terminalPrint("Z is not a number")
            allValuesNumeric = False
        if not re.match(pattern, Rx):
            self.root.terminalPrint("Rx is not a number")
            allValuesNumeric = False
        if not re.match(pattern, Ry):
            self.root.terminalPrint("Ry is not a number")
```

```
606             allValuesNumeric = False
607         if not re.match(pattern, Rz):
608             self.root.terminalPrint("Rz is not a number")
609             allValuesNumeric = False
610 
611         #If match succesful for all
612         if allValuesNumeric:
613             self.root.terminalPrint("All values numeric, sending ML command")
614             #Build Position
615             commandPos = Position(float(x),float(y),float(z),float(Rx),float(Ry),float(Rz),None)
616             self.root.statusPrint("Sending ML command")
617             #Thread so that command doesn't interupt UI
618             MJThread = threading.Thread(target=self.sendMJ, args=[commandPos, self.moveParameters], name="MJ Command Thread")
619             MJThread.start()
620         else:
621             self.root.statusPrint("ML command not sent due to a value not being a number")
622 
623     #Load filament by extruding a set amount
624     def loadFilament(self):
625         if self.checkIfBusy(message="Load Filament"):
626             return
627         self.root.statusPrint("Loading filament...")
628         #Adjusted based on multiplier because the length is cool not heated length
629         if not self.root.coolendMode:
630             lengthToLoad = self.loadLength/self.heatedFilamentMultiplier
631         else:
632             lengthToLoad = self.loadLength
633         loadThread = threading.Thread(target=self.extrude, args=[lengthToLoad], kwargs={"timeoutMultiplier":5},name="Load Filament Thread")
634         loadThread.start()
635 
636     #Unload filament by extruding a negative amount
637     def unloadFilament(self):
638         if self.checkIfBusy(message="Unload Filament"):
639             return
640         self.root.statusPrint("Unloading filament...")
641          #Adjusted based on multiplier because the length is cool not heated length
642         if not self.root.coolendMode:
643             lengthToLoad = self.loadLength/self.heatedFilamentMultiplier
644         else:
645             lengthToLoad = self.loadLength
646         unloadThread = threading.Thread(target=self.extrude, args=[-lengthToLoad], kwargs={"timeoutMultiplier":5,"moveParameters":self.unloadPara
647         unloadThread.start()
648 
649 
650     def extrudeButton(self):
651         # Read the values from each entry box
652         extrudeRate = self.root.J7CoordEntry2.get()
653         # Check if any values are blank
654         pattern = r"^-?(\d+(?:\.\d+)?)"  # Regular expression for a valid float
655         allValuesNumeric = True
656 
657         if not re.match(pattern, extrudeRate):
658             self.root.terminalPrint("J7 is not a number")
659             allValuesNumeric = False
660 
661         if allValuesNumeric:
662             self.root.terminalPrint("All values numeric, sending extrude command")
663             extrudeThread = threading.Thread(target=self.extrude, args=[float(extrudeRate)], name="Extrude Thread")
664             extrudeThread.start()
665 
666         else:
667             self.root.statusPrint("Extrude command not sent due to a value not being a number")
668 
669     def zeroJ7(self):
670         self.root.serialController.sendSerial("Z7\n")
671         self.waitForResponseAndProcess("POSZ7",15,message="Zero J7",setFlag=False)
672 
673     #Move to the neutral position where all joints are at 0 degrees
674     def prepMoveHome(self):
675         if self.checkIfAllBusy(message="Home"):
676             return
677         #start thread
678         moveHomeThread = threading.Thread(target=self.moveHome, name="Move Home Thread")
679         moveHomeThread.start()
680 
681     #Move to the position where the arm rests on itself
```

```
    def prepMoveSafe(self):
        #start thread
        moveSafeThread = threading.Thread(target=self.moveSafe, name="Move Safe Thread")
        moveSafeThread.start()

    #set move parameters to open loop
    def setOpenLoop(self):
        self.moveParameters.setLoopMode(1)
        self.root.printController.defaultPrintParameters.setLoopMode(1)
        self.root.loopStatus.config(text="Open Loop")

    #set move parameters to closed loop
    def setClosedLoop(self):
        self.moveParameters.setLoopMode(0)
        self.root.printController.defaultPrintParameters.setLoopMode(0)
        self.root.loopStatus.config(text="Closed Loop")

    #Get the joint colors, green means near zero, red means near limit switch
    def getJointColors(self,J1,J2,J3,J4,J5,J6):
        if J1 == None:
            return ["#000000"]*6
        Js = [float(x) for x in [J1,J2,J3,J4,J5,J6]]
        Js = np.matrix(Js).T
        Limits = np.matrix([self.J1Limits,self.J2Limits,self.J3Limits,self.J4Limits,self.J5Limits,self.J6Limits])

        LimitsNeg = Limits[:,0]
        LimitsPos = Limits[:,1]
        #Find Angles Closest to Limits rather than furthest from center
        NegativeDeltas = LimitsNeg-Js
        PositiveDeltas = LimitsPos-Js
        Scores = np.zeros([6,1])
        for i in range(len(Js)):
            if abs(PositiveDeltas[i]) < abs(NegativeDeltas[i]):
                if PositiveDeltas[i]<LimitsPos[i]:
                    Scores[i] = (1-abs(PositiveDeltas[i])/LimitsPos[i])*255
                else:
                    Scores[i] = 0
            else:
                if NegativeDeltas[i]>LimitsNeg[i]:
                    Scores[i] = (1-abs(NegativeDeltas[i])/abs(LimitsNeg[i]))*255
                else:
                    Scores[i] = 0
        greenArray = 255-Scores
        green = [int(x[0])for x in greenArray.astype(int)]
        red = [int(x[0])for x in Scores.astype(int)]
        blue = 0
        jointColors = [" "]*6
        for i in range(len(green)):
            rgb = (red[i],green[i],blue)
            jointColors[i] = '#%02x%02x%02x' % rgb
        return jointColors

    #endregion GUI
    #region Move Commands ================================

    #For extrusion without movement
    #timeoutMultiplier is how much extra time per expected second is needed
    def extrude(self,extrudeRate,moveParameters=None,relative=True,timeoutMultiplier=3):
        #Check whether in coolend mode and if the target temp is reached
        if not self.root.temperatureController.HotendTargetReached() and not self.root.coolendMode:
            self.root.statusPrint("Cannot extrude. Hotend target temperature not reached.")
            self.root.printController.flag = "Hotend target temperature not reached"
            return
        elif self.root.coolendMode:
            if not self.root.printController.ignoreFlags:
                self.root.warningPrint("Extruding in coolend mode. Extruder should not be attached to the hotend else damage will occur.")

        timeoutMin = 8 #minimum timeout
        #If no parameters set to default
        if moveParameters is None:
            moveParameters = self.defaultExtrudeParameters

        #if in cool end mode or extrusion is less than 0
        #For large multipliers, the negative extrusion launches the filament out the back
        if self.root.coolendMode or extrudeRate < 0:
            #Convert extruderate to J7 degrees
```

```
            J7 = extrudeRate*self.extruder_deg_per_mm_cool
        else:
            #Convert extruderate to J7 degrees
            J7 = extrudeRate*self.extruder_deg_per_mm

        timeout = timeoutMultiplier*abs(extrudeRate)/moveParameters.speed + timeoutMin #timeout is proportional to amount of extrusion

        if self.checkIfBusy(message="E7 extrude"):
            return
        #Build command
        command = MoveCommand("E7",None,moveParameters=moveParameters,J7=J7,J7Rel=relative)
        self.root.serialController.sendSerial(str(command))#convert command to string before sending
        self.waitForResponseAndProcess("POSE7",timeout=timeout, message="E7 extrude")


    #Move to a xyz coordinate in best path, nonlinear path
    #MJ cannot control extruder
    def sendMJ(self,commandPos, moveParameters,timeout=30):

        #Leaving this specific check for awaitingMoveResponse so user knows why move failed
        #Even though nominal check handles this
        if self.awaitingMoveResponse:
            self.root.printController.flag = "Already moving"
            self.root.statusPrint("Cannot send ML command as currently awaiting response from a previous move command")
            return
        #self.root.terminalPrint("Sending MJ command...")

        #Nominal check
        if self.notNominalCheck(message="Send MJ"):
            self.root.printController.flag = "Arm busy"
            return

        # Taken from AR4.py
        # command =
"ML"+"X"+RUN['xVal']+"Y"+RUN['yVal']+"Z"+RUN['zVal']+"Rz"+rzVal+"Ry"+ryVal+"Rx"+rxVal+"J7"+J7Val+"J8"+J8Val+"J9"+J9Val+speedPrefix+Speed+"Ac"+AC(
        # Create the command
        command = MoveCommand("MJ",commandPos, moveParameters)
        #self.root.terminalPrint("Command to send:")
        #self.root.terminalPrint(str(command)[0:-2])

        # Send the serial command
        self.awaitingMoveResponse = True # Set the awaiting move response flag
        self.serialController.sendSerial(str(command))

        #Timeout and feedback handling
        self.waitForResponseAndProcess("POSMJ",timeout=timeout,message="Send MJ")
        self.awaitingMoveResponse = False

    #Move linear to a xyz coordinater, timeout default is 5
    def sendML(self, pos, moveParameters, extrudeRate=0, RelativeExtrude=True,timeout=30):

        #simply adjustment based on speed for the standard timeout
        if moveParameters.speedType == "m" and timeout == 30:
            #assume a move length of 50
            timeout2 = 50/moveParameters.speed
            if timeout2 > timeout:
                timeout = timeout2
        if self.awaitingMoveResponse:
            self.root.printController.flag = "Already moving"
            self.root.statusPrint("Cannot send ML command as currently awaiting response from a previous move command")
            return
        #self.root.terminalPrint("Sending ML command...")
        # Taken from AR4.py, line XXXX
        # command =
"ML"+"X"+RUN['xVal']+"Y"+RUN['yVal']+"Z"+RUN['zVal']+"Rz"+rzVal+"Ry"+ryVal+"Rx"+rxVal+"J7"+J7Val+"J8"+J8Val+"J9"+J9Val+speedPrefix+Speed+"Ac"+AC(

        # Check if board is not connected or arm is not calibrated
        if self.notNominalCheck(message="send ML"):
            self.root.printController.flag = "Arm busy"
            return

        #Check if extruding and hotend temp is reached if extrusion is requested
        if not self.root.temperatureController.HotendTargetReached() and not self.root.coolendMode and extrudeRate != 0:
            self.root.statusPrint("Cannot extrude. Hotend target temperature not reached.")
            self.root.printController.flag = "Hotend target temperature not reached"
            return
```

```
832
833         #Convert extrudeRate to change in motor angle
834         J7 = extrudeRate*self.extruder_deg_per_mm #convert from mm to deg
835         # Create the command
836         command = MoveCommand("ML",pos, moveParameters, J7=J7, J7Rel=RelativeExtrude)
837         #self.root.terminalPrint(str(command)[0:-2])
838
839         # Send the serial command
840         self.awaitingMoveResponse = True # Set the awaiting move response flag
841         self.serialController.sendSerial(str(command))
842
843         #Timeout and feedback handling
844         self.waitForResponseAndProcess("POSML",timeout=timeout,message="Send ML")
845         self.awaitingMoveResponse = False
846
847     #Move arm joint angles
848     def sendRJ(self, Joints, moveParameters, timeout=30):
849         #self.root.terminalPrint("Sending RJ command...")
850         if self.awaitingMoveResponse:
851             self.root.statusPrint("Cannot send ML command as currently awaiting response from a previous move command")
852             self.root.printController.flag = "Already moving"
853             return
854         # Create the command
855         # RJA0B0C0D0E0F0J70J80J90Sp25Ac10Dc10Rm80WNLm000000
856         command = MoveCommand("RJ",Joints, moveParameters)
857         # Check if a board is connected or if the arm is not calibrated
858         if self.notNominalCheck(message="send RJ"):
859             self.root.printController.flag = "Arm busy"
860             return
861         #print("Command is ", str(command))
862         # Send the serial command
863
864         self.awaitingMoveResponse = True # Set the awaiting move response flag
865         self.serialController.sendSerial(str(command))
866
867         #Timeout and feedback handling
868         self.waitForResponseAndProcess("POSRJ",timeout=timeout,message="Send RJ")
869         self.awaitingMoveResponse = False
870
871     #Waits for response from serial without starting a thread then calls process position
872     def waitForResponseAndProcess(self,prefix,timeout=15,message="", setFlag = True):
873         #Wait for the response from the serial controller with the given prefix and timeout
874         response = self.root.serialController.waitForResponse(prefix,timeout)
875
876         if response is not None:
877             #If the response error rather than timed out
878             if response[:5] == "Error":
879                 self.root.printController.flag = response
880                 self.root.terminalPrint(response + " stopped serial response processing")
881                 return
882             self.processPosition(response)
883             if self.root.DebugMode:
884                 self.root.terminalPrint(response)
885                 self.root.statusPrint("Move command executed successfully")
886         else:
887             #Set flag used whether the timeout should send a flag to the printer
888             if setFlag:
889                 self.root.printController.flag = "timeout after: " + str(timeout) + "s"
890             self.root.terminalPrint(message + " timed out after "+str(timeout)+ "s")
891
892     #syncs moveparameters with print parameters if enabled
893     def sync(self):
894         if self.syncWithPrintParameters:
895             if self.root.DebugMode:
896                 self.root.statusPrint("Parameters synced with print parameters")
897             self.moveParameters = self.root.printController.defaultPrintParameters
898
899     #These are teensy commands that have not been implemented
900     def moveCircle(self):
901         pass
902     def moveArc(self):
903         pass
904
905     #Estimate movetime so timeout can be reasonable, only for ML command
906     #Use curPos given to it not the global curPos
907     def estimateMoveTime(self,lastPos,curPos,speed):
```

```
        #Calculate delta position
        delX = curPos.x-lastPos.x
        delY = curPos.y-lastPos.y
        delZ = curPos.z-lastPos.z
        #pythagorean theorem
        distance = math.sqrt(delX**2+delY**2+delZ**2)
        #T=D/r
        estimateTime = distance/speed
        if self.root.PrintDebugMode:
            print("Time:", estimateTime, " distance:", distance, " speed:", speed)
        return estimateTime

    # Moves the robot to a safe position to be turned off
    # Cancels everything, ignore flags, and just sends the serial command
    # timeout is large because movesafe can be held in the teensy queue
    def moveSafe(self,timeout=60):
        #This is important because move safe will not move safe if arm is not calibrated
        if self.serialController.boardConnected is False or self.armCalibrated is False:
            self.root.statusPrint("Failed to move to safe positiion, Board not connected")
            return
        #Pause print so everything stops moving
        #It is assume the user will want the print paused if pressing this button
        self.root.printController.pauseAll()
        self.cancelActions()#pause actions like calibration or testing
        #wait until commands have finished

        #J1-J6 joints that put the arm in a safe position
        SafePositionJoints = (0, -40, 60, 0, -45, 0)

        #Build RJ command
        command = MoveCommand("RJ",SafePositionJoints, self.defaultMoveParameters)

        #Will send regardless if a move is in progress
        self.serialController.sendSerial(str(command))
        self.waitForResponseAndProcess("POSRJ",timeout=timeout)

    # Moves to the neutral position, all joints at zero degrees
    def moveHome(self):
        if self.awaitingMoveResponse:
            self.root.statusPrint("Cannot send HM command as currently awaiting response from a previous move command")
            self.root.printController.flag = "Already moving"
            return
        # Check if a board is connected or if the arm is not calibrated
        if self.notNominalCheck(message="move home"):
            self.root.printController.flag = "Arm busy"
            return
        self.awaitingMoveResponse = True # Set the awaiting move response flag
        self.root.serialController.sendSerial("HM\n")
        self.waitForResponseAndProcess("POSHM",30,message="Move Home",setFlag=True)
        self.awaitingMoveResponse = False

    #endregion move commands

    #region ======Testing============
    def toggleLimitTest(self):
        if self.serialController.boardConnected is False:
            self.root.statusPrint("Failed to start limit switch test. No board is connected")
        # Check if we are already testing limit switches
        elif self.testingLimitSwitches:
            self.finishTest = True # Set the flag to finish the test
            self.root.limitTestButton.configure(relief="raised") # Make the button look un-toggled
        # If not, check if we are busy with anything else (self.testingLimitSwitches must be False if here)
        elif self.checkIfAllBusy():
            self.root.statusPrint("Failed to start limit switch test. Arm is busy.")
        # If we reach this, the arm is not busy with anything and we can start the test
        else:
            self.testingLimitSwitches = True # set the flag
            self.root.limitTestButton.configure(relief="ridge") # Make the button look toggled
            self.root.statusPrint("Starting limit switch test")
            limitSwitchThread = threading.Thread(target = self.limitTest, name="Limit Switch Test Thread")
            limitSwitchThread.start()

    def limitTest(self):
        #Will send command if response is not given in 2 seconds
        while not self.finishTest and self.testingLimitSwitches:
            self.serialController.sendSerial("TL\n") # Send instruction
```

```
            response = self.root.serialController.waitForResponse("TL",2)
            # Check if serial controller has a response ready
            if response is not None:
                # If so, read it in
                #self.root.terminalPrint("testing limit switch "+response)
                # Limit switch test will never return an error so we can always directly process
                self.root.J1LimState.config(text=response[response.find('J1')+5:response.find("   J2")].strip())
                self.root.J2LimState.config(text=response[response.find('J2')+5:response.find("   J3")].strip())
                self.root.J3LimState.config(text=response[response.find('J3')+5:response.find("   J4")].strip())
                self.root.J4LimState.config(text=response[response.find('J4')+5:response.find("   J5")].strip())
                self.root.J5LimState.config(text=response[response.find('J5')+5:response.find("   J6")].strip())
                self.root.J6LimState.config(text=response[response.find('J6')+5:].strip())
                # Check if the user wants to stop the test
                # This is done in here to prevent the program eternally waiting on a response
        self.testingLimitSwitches = False # Reset the test flag
        self.finishTest = False # Reset the finish testing flag
        self.root.statusPrint("Stopping limit switch test")

    def toggleEncoderTest(self):
        if self.serialController.boardConnected is False:
            self.root.statusPrint("Failed to start encoder test. No board is connected")
        # Check if we are already testing encoders
        elif self.testingEncoders:
            self.finishTest = True # Set the flag to finish the test
            self.root.encoderTestButton.configure(relief="raised") # Make the button look un-toggled
        # If not, check if we are busy with anything else (self.testingEncoders must be False if here)
        elif self.checkIfAllBusy():
            self.root.statusPrint("Failed to start encoder test. Arm is busy.")
        # If we reach this, the arm is not busy with anything and we can start the test
        else:
            self.testingEncoders = True # set the flag
            self.root.encoderTestButton.configure(relief="ridge") # Make the button look toggled
            self.root.statusPrint("Starting encoder test")
            # Send a "Set Encoder" instruction to set all values to 1000
            # I don't understand the reasoning behind this but this is what the AR4's encoder test code does so it is being included
            #self.serialController.sendSerial("SE\n") # Commented out for now for true encoder values
            # The "Set Encoder" instruction returns a "Done" except it is done with a print instead of println
            # which makes it so the serial can only be read by a "read" instead of "readline".
            # Therefore, we forcibly tell the serialController that it isn't waiting for a response
            encoderTestThread = threading.Thread(target=self.encoderTest, name="Encoder Test Thread")
            encoderTestThread.start()

    def encoderTest(self):
        #while test has not finished
        #If has reponse is not recieved another command will be sent after 2 seconds
        while not self.finishTest and self.testingEncoders:
            #Send the command
            self.serialController.sendSerial("RE\n") # Send instruction
            #wait 2 seconds for a response
            #send and recieve commands are RE
            response = self.serialController.waitForResponse("RE",2)
            if response is not None:
                self.root.terminalPrint(response)
                # Encoder test will never return an error so we can always directly process
                self.root.J1EncState.config(text=response[response.find('J1')+5:response.find("   J2")].strip())
                self.root.J2EncState.config(text=response[response.find('J2')+5:response.find("   J3")].strip())
                self.root.J3EncState.config(text=response[response.find('J3')+5:response.find("   J4")].strip())
                self.root.J4EncState.config(text=response[response.find('J4')+5:response.find("   J5")].strip())
                self.root.J5EncState.config(text=response[response.find('J5')+5:response.find("   J6")].strip())
                self.root.J6EncState.config(text=response[response.find('J6')+5:].strip())
        self.testingEncoders = False # Reset the test flag
        self.finishTest = False # Reset the finish testing flag
        self.root.statusPrint("Stopping encoder test")

    #endregion testing

    #region Jogs
    def startToolJog(self):
        if self.checkIfAllBusy(message="tool jog"):
            return
        #Abbreviate for convenience
        MP = self.defaultMoveParameters
        #Vector will change a tool axis of the arm
        #1st number is which axis 1-6
        #2nd number is direction 0 negative, 1 positive
        #V = "10"
```

```
        V = str(self.V1)+str(self.V2) #concatenate values
        #Note acceleration ramp don't matter because they're overriden
        command = f"LTV{V}S{MP.speedType}{MP.speed}Ac{MP.acceleration}Dc{MP.deceleration}Rm{MP.ramp}WFLM{MP.loopMode*6}\n"
        self.root.serialController.sendSerial(command)
        #self.root.serialController.sendSerial("start") #Don't know why it needs another command to start

    #select tool jog axis using the axis specified in the UI
    def selectToolJogAxis(self,axis):
        self.V1 = axis
        self.root.terminalPrint(f"Jog set to axis {axis}")

    #change tool jog direction
    def changeToolJogDirection(self):
        self.V2 ^= 1 #XOR bit shift
        if self.V2:
            dir = "+"
        else:
            dir = "-"
        self.root.toolJogDirection.config(text=f"Direction: {dir}")

    #Stop tool jog by sending "S"
    def stopToolJog(self):
        if self.checkIfAllBusy():
            return
        command = "S"
        self.root.serialController.sendSerial(command)

    def startCartesianJog(self):
        if self.checkIfAllBusy():
            self.root.terminalPrint("Arm busy")
        pass
    def stopCartesianJog(self):
        if self.checkIfAllBusy():
            self.root.terminalPrint("Arm busy")
        pass
    def startJointJog(self):
        if self.checkIfAllBusy():
            self.root.terminalPrint("Arm busy")
        pass

    #This is the other type of tool jog, not sure what it does
    def offsetTool(self):
        if self.checkIfAllBusy():
            self.root.terminalPrint("Arm busy")
        pass

    #endregion Jogs

    #region Other Functions

    # Checks if any of the flags relating to the arm performing a task are True and if so, return True
    # This is the minimum check does not consider if board is connected or arm is calibrated
    # A message is optional to streamline error messages
    def checkIfBusy(self, message = None, display = True):
        value = self.calibrationInProgress or self.awaitingMoveResponse or self.testingLimitSwitches or self.testingEncoders or self.awaitingPosR
        if value:
            if message and display:
                self.root.terminalPrint("Cannot " + message + ", Arm busy")
            elif display:
                self.root.terminalPrint("Arm busy")
        return value

    # meant to add calibration check without printing check
    # so move commands can execute while printing
    # returns true if everything is not nominal ie. something is busy or not connected
    def notNominalCheck(self, message = None, display = True):
        value = self.checkIfBusy(message=message,display=display)
        if self.root.serialController.boardConnected is False:
            self.root.warningPrint("No board connected")
            value = True
        elif self.armCalibrated is False:
            self.root.warningPrint("Arm not calibrated cannot proceed")
            value = True
        return value

    #Checks if anything is busy
```

```
    #Do not use this command on calibrate and it is assumed that these check do not need to be done to start calibration
    def checkIfAllBusy(self,message=None, display=True):
        value = False
        if self.notNominalCheck(message=message,display=display):
            value = True
        #check printer for printing processes
        if self.root.printController.checkIfPrinterBusy(message=message,display=display):
            value = True
        return value

    #This should reset everything in check if all busy
    def reset(self):
        #Reset all flags
        self.awaitingMoveResponse = False
        self.calibrationInProgress = False
        self.awaitingPosResponse = False
        self.testingLimitSwitches = False
        self.testingEncoders = False
        #Pause print if printing
        if self.root.printController.printing:
            self.root.printController.pausePrint()
        self.root.printController.bedCalibration = False
        self.root.printController.cornerSweeping = False

        time.sleep(0.5) # Small delay to ensure all flags are reset before killing threads

        running_threads = threading.enumerate() # Get a list of all running threads
        #Print thread for debugging
        self.root.terminalPrint("Running threads:"+ ", ".join([thread.name for thread in running_threads])) # Print the list of running threads t

        killingThreads = False #whether there are threads to kill
        #For each running thread
        for thread in running_threads:
            #if the thread is not the current thread and is not in the list of unstoppable threads, join it (kill it)
            if thread is not threading.current_thread() and thread.name not in self.root.unstoppableThreads:
                killingThreads = True
                break

        if killingThreads:
            #Notify user to not interrupt the reset process because threads are being killed
            self.root.warningPrint("Attempting to kill threads. Press ok then please wait")
            self.root.statusPrint("Please wait...")
            #This is what actually kills the threads, it will raise an error to stop waiting for serial
            #The only way it can truly kill thread is to stop serial waiting otherwise it will just wait for it to finish
            self.root.serialController.RaiseError("Reset")
            for thread in running_threads:
                if thread is not threading.current_thread() and thread.name not in self.root.unstoppableThreads:
                    self.root.terminalPrint(f"Killing thread: {thread.name}")
                    #wait for thread to finish
                    self.root.join(thread)
                    self.root.terminalPrint(f"Thread {thread.name} killed")
            self.root.statusPrint("Reset complete")
        else:
            self.root.statusPrint("No Threads Running. Reset complete")

        #Clear queue because response from arm will not matter if things were cancelled
        self.root.serialController.clearQueue()
        #Reset the printer flag
        self.root.printController.flag = None

    #cancel all actions except moveresponse so moves can terminate properly
    def cancelActions(self):
        self.calibrationInProgress = False
        self.testingLimitSwitches = False
        self.testingEncoders = False

    #endregion other functions
    #region ========|Origin|==================
    def setOrigin(self,origin=None):

        #origin can be set by an origin or at current position
        if origin is None or origin.x is None:
            if self.checkIfAllBusy(message="set origin"):
                return
            self.requestPositionAndWait #requests and waits
            self.origin.setOrigin(self.curPos)
```

```
            self.curPos.origin = self.origin
            self.root.xCurCoordOrigin.config(text=self.curPos.x)
            self.root.yCurCoordOrigin.config(text=self.curPos.y)
            self.root.zCurCoordOrigin.config(text=self.curPos.z)
        #if origin is not none, no checks because the arm is setting the origin itself
        else:
            self.origin=copy.deepcopy(origin)
            self.curPos.origin = self.origin
            self.root.xCurCoordOrigin.config(text=self.origin.x)
            self.root.yCurCoordOrigin.config(text=self.origin.y)
            self.root.zCurCoordOrigin.config(text=self.origin.z)

    #Moves to current set origin
    def moveOrigin(self):
        if self.origin.checkOriginSet():
            moveParameters = copy.deepcopy(self.defaultMoveParameters)
            moveParameters.wrist = "N" #Make wrist condition J4 near 0
            self.sendMJ(Position(self.origin.x,self.origin.y,self.origin.z,0,self.root.printController.defaultAngle,0, None), moveParameters=move


    #Updates UI display of UI from origin, called whenever a position is received
    def updateDeltaFromOrigin(self):
        if not self.origin.checkOriginSet():
            self.root.statusPrint("Origin not set")

        [deltaX,deltaY,deltaZ] = self.curPos.GetRelative()[:3]

        deltaX = round(deltaX,2)
        deltaY = round(deltaY,2)
        deltaZ = round(deltaZ,2)

        self.root.xDeltaOrigin.config(text=deltaX)
        self.root.yDeltaOrigin.config(text=deltaY)
        self.root.zDeltaOrigin.config(text=deltaZ)

    #Move to the default (recommended) origin
    def moveRecommendedOrigin(self):
        threading.Thread(target=self.sendMJ, args=(self.root.printController.recommendedOriginPosition, self.defaultMoveParameters),name= "Move [

    #endregion origin

#region--- Other Classes ---
class Position:
    #Takes an origin object for each position which can be relative to an origin
    #But all positions are actual relative to the arm's origin (global origin)
    def __init__(self, x, y, z, Rx, Ry, Rz, originObj):
        if x is not None and y is not None and z is not None:
            self.x = float(x)
            self.y = float(y)
            self.z = float(z)
            self.Rx = float(Rx)
            self.Ry = float(Ry)
            self.Rz = float(Rz)
        self.origin = originObj

    def GetAbsolute(self):
        return [self.x, self.y, self.z, self.Rx, self.Ry, self.Rz]

    def GetRelative(self):
        if self.origin is not None and self.origin.originSet == True:
            relX = self.x - self.origin.x
            relY = self.y - self.origin.y
            #print("test for z and origin of z", self.z,self.origin.z)
            relZ = self.z - self.origin.z
            return [relX, relY, relZ, self.Rx, self.Ry, self.Rz]
        else:
            return [None*6]

    #individual get relative commands
    def GetRelativeX(self):
        if self.origin is not None and self.origin.originSet == True:
            relX = self.x - self.origin.x
            return relX
        else:
            return None
    def GetRelativeY(self):
```

```
        if self.origin is not None and self.origin.originSet == True:
            relY = self.y - self.origin.y
            return relY
        else:
            return None
    def GetRelativeZ(self):
        if self.origin is not None and self.origin.originSet == True:
            relZ = self.z - self.origin.z
            return relZ
        else:
            return None

    #Set position using relative values, will set the abolute values with consideration of the object's origin
    def SetRelative(self,relX,relY,relZ,Rx,Ry,Rz):
        #If the origin is set
        if self.origin is not None and self.origin.originSet:
            self.x = float(relX) + self.origin.x
            self.y = float(relY) + self.origin.y
            self.z = float(relZ) + self.origin.z
            self.Rx = Rx
            self.Ry = Ry
            self.Rz = Rz

    #Used to set all position values when position was initialized with None values
    def SetAbsolute(self, x, y, z, Rx, Ry, Rz):
        self.x = x
        self.y = y
        self.z = z
        self.Rx = Rx
        self.Ry = Ry
        self.Rz = Rz
    #Convert coordinates to an origin
    def toOrigin(self):
        newOrigin = Origin(self.x,self.y,self.z)
        return newOrigin

#Origin class, does not contain rotation information
class Origin:
    def __init__(self, x, y, z):
        if x is not None and y is not None and z is not None:
            self.x = float(x)
            self.y = float(y)
            self.z = float(z)
            self.originSet = True
        else:
            #origin is not set if the coordinates are null
            self.originSet = False

    def getOrigin(self):
        return [self.x, self.y, self.z]

    def setOrigin(self, position):
        self.x = float(position.x)
        self.y = float(position.y)
        self.z = float(position.z)
        self.originSet = True

    def checkOriginSet(self):
        return self.originSet

    #convert origin to a position object
    #Positions origin is itself so the relative coordinate should be 0,0,0
    def toPosition(self,angle=90):
        origin = copy.deepcopy(self)
        pos = Position(self.x,self.y,self.z,0,angle,0,origin) #standard R position
        return pos

#Move parameters information in one class
class MoveParameters:
    def __init__ (self,speed,acceleration,deceleration,ramp,loopMode,speedType,wrist="A"):
        self.speed = speed
        self.acceleration = acceleration
        self.deceleration = deceleration
        self.ramp = ramp
        self.loopMode = loopMode
        self.speedType = speedType
```

```
        self.wrist = wrist
    def setLoopMode(self,loopMode):
        self.loopMode = loopMode
    def getLoopMode(self):
        return self.loopMode

#move command information to set up any move command, does not send a move command
#can be converted to a string and returns the command that will be setn on serial
class MoveCommand:
    def __init__(self,type,jointsOrPosition, moveParameters, J7=0, J7Rel=False):
        self.jointsOrPosition = jointsOrPosition
        if isinstance(jointsOrPosition, Position):
            self.A = jointsOrPosition.x
            self.B = jointsOrPosition.y
            self.C = jointsOrPosition.z
            self.F = jointsOrPosition.Rx
            self.E = jointsOrPosition.Ry
            self.D = jointsOrPosition.Rz
        #No parameters for E7 besides J7
        elif type == "E7":
            pass
        else:
            self.A = jointsOrPosition[0]
            self.B = jointsOrPosition[1]
            self.C = jointsOrPosition[2]
            self.D = jointsOrPosition[3]
            self.E = jointsOrPosition[4]
            self.F = jointsOrPosition[5]
        if J7 != None:
            self.J7 = J7
            #convert to binary true/false
            self.J7Rel = 1 if J7Rel else 0
        else:
            self.J7 = 0
            self.J7Rel = 0
        self.moveParameters = moveParameters
        self.type = type
        #debugging print("type is: ",self.type, type)
    def __str__(self):
        command = "Error"#If no command is made
        if self.type=="ML":
            command = f"MLX{self.A}Y{self.B}Z{self.C}Rz{self.D}Ry{self.E}Rx{self.F}J7{self.J7}Rel{self.J7Rel}J80.00J90.00S{self.moveParameters.sp
{self.moveParameters.speed}Ac{self.moveParameters.acceleration}Dc{self.moveParameters.deceleration}Rm{self.moveParameters.ramp}Rnd0W{self.movePar
        elif self.type=="MJ":
            command = f"MJX{self.A}Y{self.B}Z{self.C}Rz{self.D}Ry{self.E}Rx{self.F}J7{self.J7}J80.00J90.00S{self.moveParameters.speedType}
{self.moveParameters.speed}Ac{self.moveParameters.acceleration}Dc{self.moveParameters.deceleration}Rm{self.moveParameters.ramp}Rnd0W{self.movePar
        elif self.type=="RJ":
            command = f"RJA{self.A}B{self.B}C{self.C}D{self.D}E{self.E}F{self.F}J7{self.J7}J80.00J90.00S{self.moveParameters.speedType}
{self.moveParameters.speed}Ac{self.moveParameters.acceleration}Dc{self.moveParameters.deceleration}Rm{self.moveParameters.ramp}WNLm{self.movePara
        elif self.type=="E7":
            command = f"E7E{self.J7}Rel{self.J7Rel}S{self.moveParameters.speedType}{self.moveParameters.speed}Ac{self.moveParameters.acceleratio
        return command
#endregion other classes
```